

# Racing Lap Time Analyzer

Technical Report

CIS126 — Computer Architecture & Organization

Option 5 — MIPS Application Program

Vatsal Sagar | Spring 2026

Montgomery County Community College | Prof. K. Mousley

## 1. Executive Summary

---

My capstone project is a MIPS assembly program called Racing Lap Time Analyzer, which analyzes lap times from a Formula 1 race. Instead of choosing a generic topic, I wanted something that actually interested me, so I based the project around racing data because I used to be a professional kart racer myself and always wanted to become a F1 driver.

The program stores lap times for four drivers across five laps each in the .data segment. It then calculates each driver's total and average lap time, finds the fastest and slowest individual laps, sorts the drivers based on total race time, and prints everything in a structured report.

The program works as expected. For example, Hamilton ends up with a total of 418.3 seconds, and the fastest lap is 82.9 seconds. I also included a simple performance analysis using Amdahl's Law, which shows a theoretical speedup of about 1.95x if the program were parallelized across four cores.

Some of the main design choices I made were storing lap times in a row-major 2D array, sorting an index array instead of moving the actual data, and representing times as integers in tenths of seconds. These choices made the program easier to manage and avoided unnecessary complexity.

## 2. Architecture Overview

---

### 2.1 How the program is structured

There are eleven procedures in total. Main sits at the top and just calls everything else in a fixed order. The computation layer underneath it handles all the number crunching, the sort sits on its own, and then there is an output layer at the bottom. Three small helper procedures handle the actual printing and get called from within the output layer.

| Procedure       | Calls                | Category    | Stack frame |
|-----------------|----------------------|-------------|-------------|
| Main            | all 9 below          | Entry point | 4 bytes     |
| computeTotals   | none                 | Computation | 12 bytes    |
| computeAverages | none                 | Computation | 8 bytes     |
| findFastestLap  | none                 | Analysis    | 12 bytes    |
| findSlowestLap  | none                 | Analysis    | 12 bytes    |
| sortDrivers     | none                 | Sort        | 16 bytes    |
| printReport     | printTime, printLaps | Output      | 12 bytes    |
| printHighlights | printTimeInline      | Output      | 4 bytes     |
| printTime       | none                 | Helper      | 4 bytes     |
| printLaps       | none                 | Helper      | 12 bytes    |
| printTimeInline | none                 | Helper      | 4 bytes     |

### 2.2 The two data structures

Everything lives in the .data segment. There are two main structures.

Name pointer array

nameAddr is just four words, each one a memory address pointing to a driver name string. To use it you load the base address, compute the driver index times four as a byte offset, and dereference the pointer with lw to get the actual string. It is

straightforward pointer indirection but it was my first time doing it in MIPS so it took a couple of tries to get right.

```
.data
name0:  .asciiz "Hamilton"
name1:  .asciiz "Verstappen"
name2:  .asciiz "Leclerc"
name3:  .asciiz "Norris"
nameAddr: .word name0, name1, name2, name3
```

## 2D lap-time array

lapTimes is twenty words laid out row-major, meaning all five laps for driver 0 come first, then driver 1, and so on. To get to any specific element you use this formula:

```
# address = base + (driver x 5 + lap) x 4
li  $t2, 5
mul $t3, $s0, $t2    # driver x NUM_LAPS
add $t3, $t3, $s1    # + lap index
sll $t3, $t3, 2      # x 4 bytes per word
la  $t4, lapTimes
add $t4, $t4, $t3
lw  $t4, 0($t4)      # lapTimes[driver][lap]
```

The times are stored as integers in tenths of seconds. So 845 is 84.5 seconds, 829 is 82.9 seconds. I mainly did this to avoid dealing with the FP co-processor, which would have made things more complicated than necessary.

On top of the two main structures there are a few smaller arrays: totalTimes and avgTimes each hold four words of computed results, rankOrder starts as {0, 1, 2, 3} and gets sorted, and there are six individual words holding the fastest and slowest lap metadata.

## 2.3 Register usage

I tried to keep register usage consistent across all eleven procedures. The table below shows what each register category is actually used for in this program.

| Register  | How I used it                                                        |
|-----------|----------------------------------------------------------------------|
| \$zero    | Comparisons, clearing values                                         |
| \$v0      | Syscall code; return value from helper procedures                    |
| \$a0      | Driver index passed into printLaps, time value passed into printTime |
| \$t0-\$t9 | Temporaries: loop bounds, address calculations, scratch values       |
| \$s0      | Outer loop counter (driver index) in most procedures                 |
| \$s1      | Inner loop counter (lap index) in the nested-loop procedures         |
| \$s2      | Tracks the min_pos index during sortDrivers                          |
| \$ra      | Saved to the stack in every procedure that calls jal                 |
| \$sp      | Decrementd at the start of each procedure, restored at the end       |

### 3. Implementation Details

---

#### 3.1 The main procedures

##### computeTotals

This is one of the simpler parts of the program. It uses two loops: the outer loop goes through each driver, and the inner loop goes through their five lap times. For each driver, it adds up all five laps and stores the total in the `totalTimes` array. The main thing I had to be careful with here was getting the address calculation right, which took a bit of trial and error at first.

##### sortDrivers

I did not want to move the actual lap time data around during sorting because that would mean shuffling the contents of a twenty-word array. Instead, `sortDrivers` sorts a separate array called `rankOrder` that starts as `{0, 1, 2, 3}`. The selection sort compares `totalTimes` values by going through `rankOrder` to find the driver index and then looking up that driver's total. When a swap happens, only the four index values in `rankOrder` change. At the end `rankOrder[0]` holds the index of the fastest driver, `rankOrder[1]` holds the second fastest, and so on.

This procedure also has the biggest stack frame in the program at 16 bytes, because it needs to preserve four registers across its work: `$ra`, `$s0`, `$s1`, and `$s2`. The selection sort itself needs two loop counters and a `min_pos` tracker, and all three need to be `$s` registers because the inner loop makes comparisons that would trash `$t` registers.

##### printTime and printTimeInline

These two handle the decimal display problem. They both receive a tenths-of-seconds integer in `$a0`. The MIPS `div` instruction puts the quotient in the `lo` register and the remainder in `hi`, so dividing by 10 gives whole seconds in `mflo` and the tenths digit in `mfhi`. Then the procedure prints the whole seconds, a literal dot, and the tenths digit as three separate pieces. 845 becomes 84.5. `printTimeInline` does exactly the same thing but skips the newline and the word `sec` at the end so it can be used inline inside the highlights output.

##### printReport

printReport is the one that ties the output together. It loops through the sorted rankOrder array and for each position it fetches the driver index, looks up the name pointer, calls printTime twice for the total and average, and calls printLaps for the five individual times. Keeping the driver index alive across all those calls required putting it in \$s1 rather than a \$t register, which is explained in the challenges section below.

### 3.2 Three problems I ran into

#### Register getting overwritten between calls

Early versions of printReport kept the driver index in \$t2. That worked fine until printTime was called, because printTime uses \$t registers freely as scratch space and overwrote \$t2 before printLaps had a chance to read it. The fix was moving the driver index to \$s1, which is callee-saved, and saving it in printReport's own stack frame. After that the value survived through all the nested calls. This was the bug that made the calling convention rules finally click for me.

#### Printing decimals without the FP co-processor

The first version of the output just printed whole seconds. That was obviously wrong for a lap time analyzer. My first instinct was to look into Coproc 1, but the setup was going to add a lot of complexity. The tenths encoding was the simpler fix. If you store 845 and divide it by 10, you get 84 remainder 5, and you can print those two numbers with a dot between them. It is not the most elegant solution but it works reliably for one decimal place and does not require any FP registers at all.

#### The sll at the end of the address formula

When I first wrote the address formula I forgot the sll at the end, which is the step that multiplies the element index by four to get the byte offset. Without it the program was loading from completely wrong addresses and getting garbage values. The output looked plausible at first because the garbage happened to be non-zero, which made it harder to spot. Stepping through computeTotals instruction by instruction in MARS and watching \$t4 in the register panel was what caught it.

## 4. Testing and Verification

---

### 4.1 How I tested it

Most of my testing was done by stepping through the program in MARS and watching how the registers changed. Once each procedure looked correct, I ran the full program and compared the output with values I calculated by hand. For example, I manually added Hamilton's lap times and checked if the program printed the same total. This helped catch a few small mistakes early on.

### 4.2 Test results

| What I checked                                                  | Outcome |
|-----------------------------------------------------------------|---------|
| Full output matches hand-calculated values for all four drivers | Pass    |
| Hamilton shows up as rank 1 (lowest total: 418.3 sec)           | Pass    |
| Hamilton lap 3 flagged as fastest (829 = 82.9 s)                | Pass    |
| Norris lap 4 flagged as slowest (858 = 85.8 s)                  | Pass    |
| Hamilton average: 4183 divided by 5 = 836, prints as 83.6       | Pass    |
| 845 prints as 84.5, 829 prints as 82.9                          | Pass    |
| \$s0, \$s1, \$s2 all back to zero after program ends            | Pass    |
| \$sp is the same value before and after main executes           | Pass    |
| Program ends cleanly, no infinite loop, MARS shows finished     | Pass    |

### 4.3 Edge cases

I tested a couple of edge cases by temporarily editing the .data values. I moved the fastest lap to position 0 and to position 4 to make sure findFastestLap was not exiting early or skipping the last iteration. I also moved the slowest lap to the last driver in the array to check findSlowestLap ran all four outer loop iterations. Both passed without needing any code changes.

With the current dataset, Hamilton finishes first at 418.3 seconds and Leclerc finishes second at 418.7 seconds. The selection sort orders drivers by ascending total race time. There is no explicit tie-breaking rule, so if two drivers had identical totals, the program would leave them in their existing relative order. A production version could add a secondary criterion for ties.

## 5. Performance Analysis

---

### 5.1 Instruction counts

I measured these using the MARS Instruction Counter under the Tools menu. You connect the tool, reset the program with F12, run it, and the counter shows the total. Getting per-procedure counts required running modified versions with only one procedure active at a time, which is a bit tedious but gave cleaner numbers.

| Procedure            | Approx. total | Stack ops | Overhead |
|----------------------|---------------|-----------|----------|
| main                 | ~22           | 2         | 9.1%     |
| computeTotals        | ~68           | 6         | 8.8%     |
| computeAverages      | ~28           | 4         | 14.3%    |
| findFastestLap       | ~82           | 6         | 7.3%     |
| findSlowestLap       | ~82           | 6         | 7.3%     |
| sortDrivers          | ~115          | 8         | 7.0%     |
| printReport          | ~160          | 6         | 3.8%     |
| printHighlights      | ~48           | 2         | 4.2%     |
| printTime (x8)       | ~8 ea         | 2         | 25.0%    |
| printLaps (x4)       | ~40 ea        | 6         | 15.0%    |
| printTimeInline (x2) | ~6 ea         | 2         | 33.3%    |

### 5.2 What the overhead numbers mean

The overhead column is the fraction of each procedure's instructions that are purely housekeeping: opening and closing the stack frame and saving and restoring registers. Those instructions do not contribute to the actual computation at all.

printTime is the most interesting case at 25% overhead. Its body is only six instructions but the frame costs two more, so a quarter of its work is just showing up and leaving. Because it gets called eight times, its cumulative stack overhead is 16 instructions. This

lines up with what we discussed in Week 7 about procedure overhead. `sortDrivers` has the biggest frame but only 7% overhead because its body is over a hundred instructions.

### 5.3 Cache behaviour of the inner loop

The most memory-intensive part of the program is the inner loop of `computeTotals`. Because I stored `lapTimes` in row-major order, each iteration of the inner loop loads from an address that is exactly four bytes higher than the previous one. A 32-byte cache line holds eight 32-bit words, so the first access in each driver row loads the entire row into cache in one miss. The next four accesses are all hits. That gives an 80% hit rate on the lap-time loads inside that loop.

If I had stored the array column-major instead, the stride between consecutive inner-loop accesses would have been five words times four bytes equals twenty bytes. That would likely cause a miss on every iteration depending on the cache size. That's why I decided to use row-major storage here.

### 5.4 Could this be vectorised

The inner loop of `computeTotals` does the same operation on every element: load a value, add it to the running total. That is a Reduce pattern. A 128-bit SIMD register holds four 32-bit values, so in theory you could load all four drivers' lap 0 values at once, add them all in one vector instruction, and do the whole accumulation in five vector operations instead of twenty scalar ones. That is a 4x improvement on the accumulation step.

To actually do that the data layout would need to change. Instead of all of driver 0's laps first, you would want all four drivers' lap 0 values first, then all four lap 1 values, and so on. That interleaved layout is worse for the scalar case but would enable the SIMD case. The current layout was the right choice given the scalar implementation.

### 5.5 Amdahl's Law

To think about how a parallel version would perform: the four computation procedures (`computeTotals`, `computeAverages`, `findFastestLap`, `findSlowestLap`) are completely independent of each other given the input data. They could all run on separate cores simultaneously. That is roughly 65% of the total work. The other 35% is `sortDrivers` and

the print procedures, which have to run in order because sorting depends on the totals being done first and printing depends on the sort being done first.

$$\begin{aligned} S(4 \text{ cores}) &= 1 / (\text{serial} + \text{parallel} / N) \\ &= 1 / (0.35 + 0.65 / 4) \\ &= 1 / (0.35 + 0.1625) \\ &= 1 / 0.5125 \\ &= 1.95x \text{ theoretical speedup} \end{aligned}$$

The 1.95x ceiling is relatively low because the serial fraction is large. If the dataset were much bigger, say fifty drivers and a hundred laps each, the computation procedures would dominate the runtime and the serial fraction would shrink, pushing the speedup higher.

## 6. Architectural Reflection

---

### 6.1 Memory layout decisions

The .data segment is laid out in the order I declared things: name strings first, then the nameAddr pointer array, then the 20-word lapTimes array, then the result arrays (totalTimes, avgTimes, rankOrder), then the six scalar words for fastest and slowest lap metadata.

The main decision was keeping lapTimes as a flat 2D array rather than a struct. A struct-per-driver approach would have grouped each driver's name, laps, total, and average together, which sounds more organised but would have made the analysis procedures worse. findFastestLap needs to scan all twenty lap values without caring which driver they belong to, and a struct layout would have scattered those values further apart in memory. The flat array keeps the scans sequential.

Sizing was straightforward. Four drivers times five laps times four bytes is 80 bytes for lapTimes. Everything fits within a small cluster of cache lines, so there is no concern about the data being spread across too much memory for this dataset size.

### 6.2 Stack frame walkthrough for sortDrivers

sortDrivers has a 16-byte frame because it needs to preserve four registers. Here is the full lifecycle of that frame:

```
# Prologue - open the frame
sortDrivers:
    addi $sp, $sp, -16 # move stack pointer down 16 bytes
    sw  $ra, 12($sp)  # save return address
    sw  $s2, 8($sp)   # save min_pos tracker
    sw  $s1, 4($sp)   # save inner loop counter j
    sw  $s0, 0($sp)   # save outer loop counter i

    # procedure body uses $s0, $s1, $s2 freely

# Epilogue - restore in reverse order
sd_done:
    lw  $s0, 0($sp)   # restore i
    lw  $s1, 4($sp)   # restore j
    lw  $s2, 8($sp)   # restore min_pos
```

```
lw  $ra, 12($sp)  # restore return address
addi $sp, $sp, 16  # move stack pointer back up
jr  $ra           # return to main
```

The restore order is the exact reverse of the save order, which is not strictly required by the hardware but makes the code easier to read and verify. After the epilogue `$sp` is back to exactly the value it had before the prologue, which is the invariant the calling convention requires.

### 6.3 Cache behaviour

As covered in section 5.3, the inner loop of `computeTotals` is the most memory-intensive part of the program and it behaves well because of the row-major layout. Consecutive iterations load addresses four bytes apart, which fits multiple elements per cache line and produces a high hit rate.

The output procedures also traverse `lapTimes` row by row and maintain the same sequential pattern. `sortDrivers` only touches the four-word `totalTimes` array, which is small enough to stay resident in cache for the entire sort.

### 6.4 SIMD potential

The innermost loop of `computeTotals` is the main SIMD candidate. It fits the Reduce pattern: one load and one add per element, five elements per driver. With a transposed data layout and 128-bit SIMD registers you could load all four drivers' values for one lap in a single instruction and accumulate them in one vector add, reducing twenty scalar operations to five vector ones.

The print procedures wouldn't really benefit from SIMD since they just output values one at a time.

### 6.5 Amdahl's Law

The 1.95x figure from section 5.5 comes from the 35% serial fraction. The main way to improve that would be to restructure the output stage. Right now `printReport` and `printHighlights` have to run in sequence because they write to the console in order. If each core instead wrote its assigned driver's output to a memory buffer and a final step

merged them in sorted order, the serial fraction would shrink to roughly 10%, which would push the theoretical speedup to about 3.1x on four cores.

## 7. Conclusion

---

### What worked

Everything I set out to build is working correctly. The program calculates totals and averages, identifies the fastest and slowest laps, sorts drivers properly, and prints a clean report. It also consistently produces the same output, which confirms that the logic is stable.

### What I would change

If I were to improve this further, the biggest change would be allowing user input instead of hard-coded data. Right now, any changes require editing the .asm file. I would also define constants for the number of drivers and laps to make the program easier to scale.

### What I learned

The two biggest things I learned from this project were how important memory layout decisions are, and how critical it is to follow calling conventions properly. Early on, I tried taking shortcuts with registers, and that caused bugs that were difficult to track down. By the end, I understood why those rules exist and how they help keep larger programs reliable.